

Machine Learning (WS25/26)

Niklas Rodenbüsch | February 23, 2026

1 Introduction

A **task** in Machine Learning (ML) refers to a specific problem that needs solving, and often involves an **agent** making a **decision** based on a **context** to achieve a specific **goal**.

1.1 Supervised & Unsupervised Learning

Definition 1.1 (Supervised Learning)

Supervised Learning uses labeled data for tasks such as Classification, Regression, and Ranking. The general supervised learning process maps input features x to a ground truth target y using a prediction model $\hat{y} = f(x)$.

Definition 1.2 (Unsupervised Learning)

Unsupervised Learning uses unlabeled data to find hidden patterns, applied in tasks like Clustering and Dimensionality Reduction.

1.2 Model Families

Definition 1.3 (Parametric Models)

Models that assume a specific functional form with a fixed number of parameters that do not grow with the dataset size (e.g., Linear/Logistic Regression, Linear SVM).

Definition 1.4 (Non-Parametric Models)

Models whose complexity and number of parameters grow dynamically with the amount of training data, having no fixed functional form (e.g., k-NN, Decision Trees, Kernel SVM, Naive Bayes).

Examples of Prediction Models

- **Linear Models:** Simple models characterized by low variance but high bias. They assume a linear relationship between the input features and the target and calculate the output as a weighted sum of (linear) input features:

$$\hat{y} = \theta_0 + \sum_{m=1}^M \theta_m x_m$$

- **Polynomial Regression:** Extends linear models by including higher-degree terms to capture non-linearities (more complex relationships).

$$\hat{y} = \sum_{j=1}^M \theta_j x^j$$

- **Decision Trees:** Highly interpretable models but prone to high variance and overfitting.
- **Neural Networks:** Highly powerful and flexible models that also exhibit high variance. Uses multiple layers of non-linear functions to model complex patterns.

1.3 Loss Functions - Overview

A loss function $\mathcal{L}$ quantifies the distance (i.e. error) between the model's prediction $\hat{y}$ and the true target y , thus measuring how well a model performs on given input data. Example loss functions include:

- **MSE (L2 Loss)**: Penalizes large errors heavily; consequently, it is very sensitive to outliers.

$$\mathcal{L}_{MSE}(y, \hat{y}) = (y - \hat{y})^2 = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **MAE (L1 Loss)**: Minimizes absolute differences; much more robust to outliers than MSE.

$$\mathcal{L}_{MAE}(y, \hat{y}) = |y - \hat{y}| = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **Huber Loss**: Combines L1 and L2 properties, acting like L2 for small errors and L1 for large errors.

For **classification tasks**, the true target y is discrete (e.g., $y \in \{0, 1\}$ for binary classification or one-hot encoded vectors for multi-class). The actual error metric is the **Misclassification Rate (MCR)**, which is non-differentiable. Furthermore, using MSE on probability outputs creates a non-convex loss surface. Instead, we use differentiable **proxy losses** that frame the problem probabilistically:

- **Misclassification Rate (MCR)**: The true evaluation metric (not used for gradient optimization), computing the ratio of incorrect predictions.

$$\mathcal{L}_{MCR}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y_i \neq \hat{y}_i]$$

- **Logistic Loss (BCE)**: The standard proxy loss for binary classification, which heavily penalizes confident but incorrect predictions.

$$\mathcal{L}_{BCE}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- **Softmax Cross-Entropy**: Generalizes BCE to multi-class problems. It measures the dissimilarity between the true one-hot encoded target distribution and the predicted probability distribution over C classes.

$$\mathcal{L}_{CE}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

- **Hinge Loss**: Used primarily for Support Vector Machines (SVMs) to enforce a maximum margin. It penalizes predictions that fall on the wrong side of the margin boundary (Note: assumes labels $y \in \{-1, 1\}$ and $\hat{y}$ is the raw model output).

$$\mathcal{L}_{Hinge}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i \hat{y}_i)$$

- **Kullback-Leibler (KL) Divergence**: Measures how one probability distribution $\hat{y}$ diverges from a reference true probability distribution y . Minimizing Cross-Entropy is mathematically equivalent to minimizing KL-Divergence.

$$D_{KL}(y || \hat{y}) = \sum_{c=1}^C y_c \log \left(\frac{y_c}{\hat{y}_c} \right)$$

1.4 Overfitting & Underfitting

Definition 1.5 (Overfitting)

Occurs when a model memorizes the training data and its noise, leading to poor generalization on unseen data. It is characterized by **High Variance** and **Low Bias**.

Detection: Validation error increases while training error decreases.

Mitigation: Add more data, reduce model complexity (by means of feature selection or regularization), ensembling

Definition 1.6 (Underfitting)

Occurs when a model is too simple to capture the underlying patterns of the data. It is characterized by **High Bias** and **Low Variance**.

Detection: Validation and training error remain large.

Mitigation: Add informative features, decrease regularization, increase model capacity

1.5 Probabilistic Interpretation & MLE

Models can be viewed probabilistically by assuming the target y is generated by a deterministic function plus random Gaussian noise:

$$y \sim \mathcal{N}(f(x; \theta), \sigma^2)$$

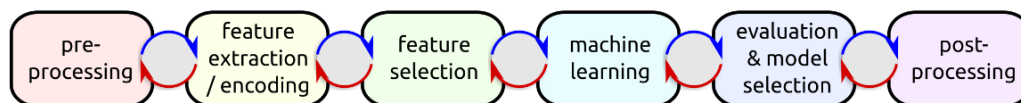
This approach acknowledges uncertainty and provides a more interpretable measure of confidence in predictions.

Definition 1.7 (Maximum Likelihood Estimation (MLE))

A statistical method used to estimate model parameters θ by maximizing the probability (likelihood) of observing the given training data under the assumed model.

For a linear regression model assuming normally distributed errors, $\epsilon_n \sim \mathcal{N}(0, \sigma^2)$, optimizing the parameters via MLE is mathematically equivalent to minimizing the Mean Squared Error (MSE).

1.6 Machine Learning Design Cycle



1. **Pre-processing:** Collecting data, cleaning, handling missing values, and addressing outliers.
2. **Feature Extraction & Encoding:** Deriving meaningful features from raw data and encoding categorical variables.
3. **Feature Selection:** Reducing dimensionality by selecting the most relevant features.
4. **Machine Learning:** Applying the chosen algorithm to train the model.
5. **Evaluation & Model Selection:** Testing performance on validation sets and tuning hyperparameters.
6. **Post-processing:** Ensuring fairness, avoiding bias, and cont. monitoring of the model after deployment.

2 Linear Methods in Machine Learning

Definition 2.1 (Linear Methods)

A foundational class of algorithms that model relationships between input features and output predictions by computing a weighted sum of the features. They assume the decision boundary is a straight line (2D) or a hyperplane (higher dimensions).

2.1 Linear Regression

Definition 2.2 (Linear Regression)

Linear regression predicts a continuous target variable by finding the optimal weights w and bias w_0 .

$$f(x_i; w) = w_0 + w_1 x_{i,1} + \dots + w_D x_{i,D} = w_0 + x_i w^T$$

It assumes a strictly linear relationship between inputs and outputs, which may fail to capture complex data structures.

Definition 2.3 (Basis Functions)

Mathematical transformations $\phi(x)$ applied to input features to map them into a higher-dimensional space, allowing linear models to capture non-linear relationships (e.g., polynomial or Gaussian functions).

$$f(x_i, w) = w_0 + \sum_{j=1}^M w_j \phi(x_i)_j$$

2.2 Optimization of Weights

To find the best weight vector w that minimizes the loss function, two main approaches can be used:

1. **Analytical Solution:** The optimal weights can be computed directly using the normal equation (iff the errors fulfill a few assumptions). This solution becomes computationally expensive for large datasets due to the matrix inversion step, which has a complexity of $O(D^3)$.

$$w = (X^T X)^{-1} X^T y$$

2. **Iterative Optimization (Gradient Descent):** A numerical method used to iteratively update weights by taking steps in the direction of the steepest descent of the loss function $J(w)$.

$$w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$$

2.3 Linear Classification & Logistic Regression

Definition 2.4 (Linear Classification)

Categorizes data into distinct classes by defining a linear decision boundary (hyperplane) that separates the feature space.

Definition 2.5 (Logistic Regression)

A linear classification model used for binary classification. It applies the **sigmoid** (or logistic) function to squeeze the linear output into a probability between 0 and 1.

$$h_w(x) = \frac{1}{1 + e^{-w^T x}}$$

2.4 Optimization in Logistic Regression

Because the sigmoid function makes the loss function non-linear, logistic regression cannot be solved analytically. It must be optimized using Gradient Descent. The gradient of the binary cross-entropy loss with respect to the weights is calculated as follows:

$$\frac{\partial J(w)}{\partial w} = \sum_{i=1}^N (h_w(x_i) - y_i) x_i$$

Definition 2.6 (Stochastic Gradient Descent (SGD))

An optimization variant that approximates the true gradient by using a single random data point or a small mini-batch at each step, making it much faster and more memory-efficient for large datasets. SGD introduces noise into the gradient estimation, which can prevent the model from getting stuck in local minima and allows for frequent weight updates without scanning the entire dataset.

3 Principles of Regularization

3.1 Bias-Variance Tradeoff

The expected test error of a model can be decomposed into three main components: Bias², Variance, and Noise:

$$\mathbb{E}_{x,y,D}[(f(x;D) - y)^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Bias: Measures how much the expected model prediction deviates from the true underlying function $f(x)$. A high bias implies the model is too simple, leading to underfitting.

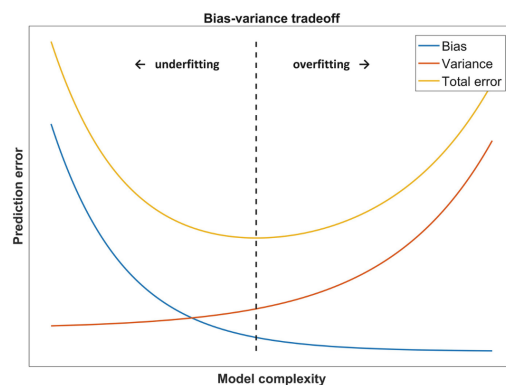
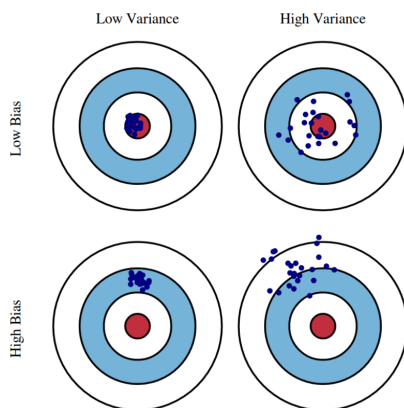
$$\text{Bias}^2 = (\bar{f}(x) - f(x))^2$$

Variance: Measures how much the model's predictions vary when trained on different datasets. A high variance implies the model is overly sensitive to training data fluctuations, leading to overfitting.

$$\text{Variance} = \mathbb{E}_D[(\hat{f}(x;D) - \bar{f}(x))^2]$$

Noise: The irreducible error inherent to the data itself due to randomness or unobserved factors.

$$\text{Noise} = \mathbb{E}_{y|x}[(y - f(x))^2]$$



3.2 Regularization Objectives

Definition 3.1 (Regularization)

A method to prevent overfitting by adding a penalty term $\Omega(\theta)$ to the learning objective, which actively discourages overly complex models.

$$\theta^* = \arg \min_{\theta} \left[\frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f(x_i; \theta)) \right] + \alpha \Omega(\theta)$$

Here α acts as the regularization strength hyperparameter. By increasing α , we slightly increase bias (simpler model) & significantly reduce variance.

3.3 Types of Regularization

1. **L1 Regularization (Lasso)**: Actively encourages **sparsity**, meaning many model weights become exactly zero during the optimization process, which effectively acts as automatic feature selection.

$$\Omega(\theta) = \frac{1}{|\theta|} \sum_{k=1}^{|\theta|} |\theta_k|$$

2. **L2 Regularization (Ridge)**: Smoothly discourages large parameter values without necessarily driving them to exactly zero.

$$\Omega(\theta) = \frac{1}{|\theta|} \sum_{k=1}^{|\theta|} \theta_k^2$$

3. **Weight Decay Equivalence**: In the context of gradient descent, applying a weight decay update rule is mathematically equivalent to optimizing a model with L2 regularization (where $\alpha = 2\mu\lambda$).

$$\theta_{t+1} = \theta_t - \alpha \theta_t - \mu \frac{\partial \mathcal{L}}{\partial \theta_t}$$

4 Support Vector Machines

Support Vector Machines (SVMs) are a powerful class of supervised learning algorithms used for classification and regression tasks.

4.1 Linear Hyperplanes & Margins

Definition 4.1 (Hyperplane)

A linear hyperplane $\mathcal{H}(w, w_0)$ is a sub-space that divides the space into two parts, defined by the equation $w^T x + w_0 = 0$. The weight vector w is always orthogonal to the hyperplane.

The distance d between a point x and the hyperplane is given by:

$$d(x, w, w_0) = \frac{|w^T x + w_0|}{\|w\|_2}$$

Maximum Margin Hyperplane: Instead of just finding any separating line, SVMs search for the hyperplane that yields the maximum margin to all data points, which improves generalization.

Scaling: Scaling w and w_0 by any non-zero scalar β does not change the hyperplane, as $\beta(w^T x + w_0) = 0$ is equivalent to $w^T x + w_0 = 0$.

Perceptron: A simple linear classifier for binary classification. An input x gets classified using the equation $f(x; w) = \text{sign}(w^T x + w_0)$.

4.2 Linear Optimization Problem

To find the maximum margin hyperplane, we want to maximize the margin $\frac{1}{\|w\|_2}$, which is mathematically equivalent to minimizing $w^T w$ (or $\frac{1}{2}w^T w$).

Hard Margin SVM (for perfectly linearly separable data) restricts the optimization with the constraint that all instances are correctly classified outside the margin.

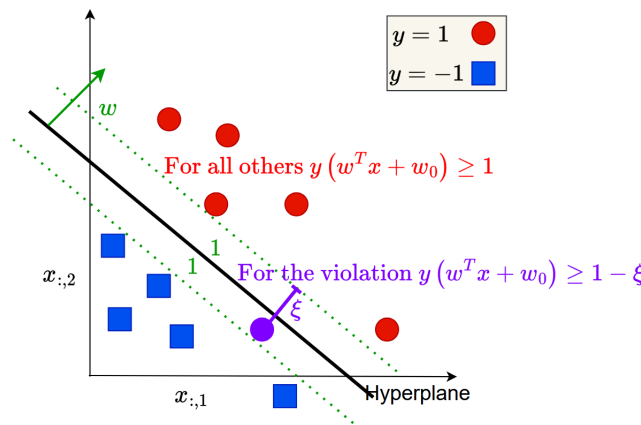
$$\arg \min_{w, w_0} \frac{1}{2} w^T w \quad \text{s.t.} \quad \forall i : y_i (w^T x_i + w_0) \geq 1$$

Scaling w and w_0 changes the functional margin without changing the actual geometric hyperplane, so we enforce the functional margin to be exactly 1 at the boundary.

4.3 Soft Margin & Hinge Loss

For data that is not perfectly linearly separable, we introduce a **slack variable** $\xi_i \geq 0$ to tolerate mistakes and violations of the margin. A hyperparameter C controls the tolerance to these violations. A higher C heavily penalizes mistakes (leading to a smaller margin), while a lower C allows more violations (leading to a wider margin).

$$\arg \min_{w, w_0} w^T w + C \sum_{i=1}^N \xi_i \quad \text{s.t.} \quad \forall i : y_i (w^T x_i + w_0) \geq 1 - \xi_i$$



The Soft Margin SVM objective can be rewritten purely as a regularized Hinge Loss optimization problem (where $\lambda = 1/C$), which can be solved using SGD.

$$\arg \min_{w, w_0} \sum_{i=1}^N \max(0, 1 - y_i (w^T x_i + w_0)) + \lambda \sum_{m=1}^M w_m^2$$

4.4 Dual Formulation & Support Vectors

Definition 4.2 (Dual Form)

While the **primal form** optimizes the parameters w and w_0 directly, the **dual form** is an equivalent optimization problem that maximizes over Lagrange multipliers α .

Definition 4.3 (Support Vectors)

The subset of training data points that lie exactly on the margin boundary or violate it. In the dual formulation, these are the only points where $\alpha_i > 0$. They completely define the decision boundary.

$$f(x, \alpha, w_0) = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i x_i^T x + w_0 \right)$$

In the dual prediction model, inference on a new point only requires computing its dot product similarity with the Support Vectors, completely ignoring all other training points.

4.5 Non-Linear SVM & The Kernel Trick

Data that is not linearly separable in its original space can become separable by applying a non-linear mapping $\phi(x)$ to project it into a higher-dimensional space.

Definition 4.4 (The Kernel Trick)

Computing the dot product $\phi(x_i)^T \phi(x_j)$ in a high-dimensional space is computationally expensive. The kernel trick replaces this dot product with a Kernel function $K(x_i, x_j)$, which computes the equivalent dot product directly in the lower-dimensional input space. The dual prediction model using a kernel becomes:

$$f(x, w, w_0) = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i K(x_i, x) + w_0 \right)$$

Common Kernels include:

- **Linear Kernel:** $K(x, x') = x^T x'$
- **Polynomial Kernel:** $K(x, x') = (x^T x' + c)^d$
- **Radial Basis Function (RBF):** $K(x, x') = e^{-\gamma \|x - x'\|^2}$

4.6 Pros & Cons

- **Pros:** Works well when there is a clear margin, highly effective in high-dimensional spaces, and very memory efficient (since it only stores support vectors).
- **Cons:** Struggles with overlapping classes, scales poorly to very large datasets (computationally expensive to train), performs poorly on extremely noisy data, and choosing the optimal kernel is difficult.

5 Decision Trees

5.1 Decision Trees as Piecewise Functions

Definition 5.1 (Decision Tree)

A non-parametric machine learning model that predicts the value of a target variable by learning simple decision rules inferred from the data features. It works by recursively partitioning the feature space into distinct regions. They can be used for both classification and regression tasks.

Every k-ary split can be represented as a sequence of binary splits, which are computationally faster and often yield better predictions. For each partitioned region R_j , the tree aggregates the target values of the data points in that region (e.g., mean for regression) to yield a constant prediction model $\hat{y}^{(R_j)}$.

The mathematical prediction model of a tree with parameters $\theta = \langle R_1, \dots, R_J, \hat{y}^{(R_1)}, \dots, \hat{y}^{(R_J)} \rangle$ uses an indicator function $\mathbb{I}(\cdot)$ that checks if x belongs to region R_j :

$$f(x; \theta) = \sum_{j=1}^J \hat{y}^{(R_j)} \cdot \mathbb{I}(x \in R_j)$$

Depth: The distance of a node from the root node.

Height: The maximum depth of any node in the entire tree.

5.2 Choosing a Split Point (CART Algorithm)

The goal of choosing a split is to minimize the impurity $H(\mathcal{D})$ of the resulting subsets $\mathcal{D}$. Purity measures how homogeneous the data points in a node are, meaning that for regression, the target values should be similar (low variance), and for classification, the class labels should be well-separated.

The Classification and Regression Trees (CART) algorithm finds split points by exhaustively searching through the mid-points of unique feature values. A split $x \leq v$ divides the dataset $\mathcal{D}$ into a left subset $\mathcal{D}^{(L)}$ and a right subset $\mathcal{D}^{(R)}$:

- **Regression Trees:** The impurity metric used is **variance**. The predicted target for a region is simply the mean target value of the split subset. The best split for regression maximizes the variance reduction:

$$|\mathcal{D}| \cdot H(\mathcal{D}) = \sum_{(x,y) \in \mathcal{D}} (y - \hat{y}(x))^2$$

- **Classification Trees:** The impurity metric used is **entropy**. The entropy of a discrete random variable V with K possible outcomes is defined as:

$$H(V) = - \sum_{k=1}^K p(v_k) \log_2 p(v_k)$$

- **Optimal Split:** The optimal split for decision trees maximizes the **Information Gain** $I(x \leq v)$:

$$I(x \leq v) = |\mathcal{D}| \cdot H(\mathcal{D}) - (|\mathcal{D}^{(L)}| \cdot H(\mathcal{D}^{(L)}) - |\mathcal{D}^{(R)}| \cdot H(\mathcal{D}^{(R)}))$$

5.3 Properties of Decision Trees

- **Pros:** Highly interpretable, handle categorical input values natively, handle unimportant features well, and are scalable for large datasets.
- **Cons:** They are heavily prone to overfitting and are deterministic (making them unsuitable for some specific ensemble methods).
- **Bias-Variance:** Because trees are very expressive and highly sensitive to small changes in training data, they are considered **High Variance** and **Low Bias** models.
- **Regularization:** To prevent a decision tree from overfitting, you can tune hyperparameters such as the maximum depth of the tree (max_depth) or the minimum number of samples required to form a leaf (min_leaf).

6 Neural Networks

6.1 Multi-Layer Perceptrons (MLP)

Definition 6.1 (Neuron)

A single computational unit that applies a non-linear activation function $g(\cdot)$ to a linear combination of its inputs.

$$h(x) = g \left(w_0 + \sum_{i=1}^M w_i x_i \right)$$

Definition 6.2 (Multi-Layer Perceptron (MLP))

A feedforward, fully-connected neural network that defines a parametric mapping $\hat{y} = f(x; w)$ composed of multiple layers of interconnected neurons.

Feedforward means that the network forms an acyclic directed graph where the signal only propagates forward, with no backward loops or intra-layer connections.

Fully-connected means that each neuron in a given layer is connected to all neurons in the subsequent layer. An MLP acts as a deep function composition: $\hat{y} = f^{(L)}(f^{(L-1)}(\dots f^{(1)}(x)))$.

6.2 Activation Functions

- **ReLU (Rectified Linear Unit):** Fast computation, prevents vanishing gradients.

$$g(z) = \max(0, z)$$

- **Sigmoid:** Squeezes the output into a range between 0 and 1. Its derivative is $\sigma(z)(1 - \sigma(z))$.

$$g(z) = \frac{1}{1 + e^{-z}}$$

- **Tanh:** Squeezes the output into a range between -1 and 1. Improves gradient flow.

$$g(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

6.3 Output Layers & Loss Functions

- **Regression** (continuous target): Uses a linear output layer (no activation function). It assumes a Gaussian distribution and is trained using the Mean Squared Error (MSE) loss.
- **Binary Classification:** Uses a Logistic Sigmoid output layer to predict the probability of the positive class. It is trained using Binary Cross-Entropy (Logistic Loss).

$$\hat{y}_i = P(y = 1|x) = \sigma(z) = \frac{e^z}{e^z + 1}$$

- **Multi-Class Classification:** Uses a Softmax output layer to normalize unnormalized log probabilities (logits) z_i into a valid probability distribution across all classes. Trained using Cross-Entropy or KL-Divergence.

$$\hat{y}_i = P(y = k|x) = \text{softmax}(z_i) = \frac{e^{z_k}}{\sum_i e^{z_i}}$$

6.4 Optimization & Backpropagation

Definition 6.3 (Chain Rule of Calculus)

A mathematical tool used to compute derivatives of composite functions. For a composite function $g(f(x))$, the derivative with respect to x is given by:

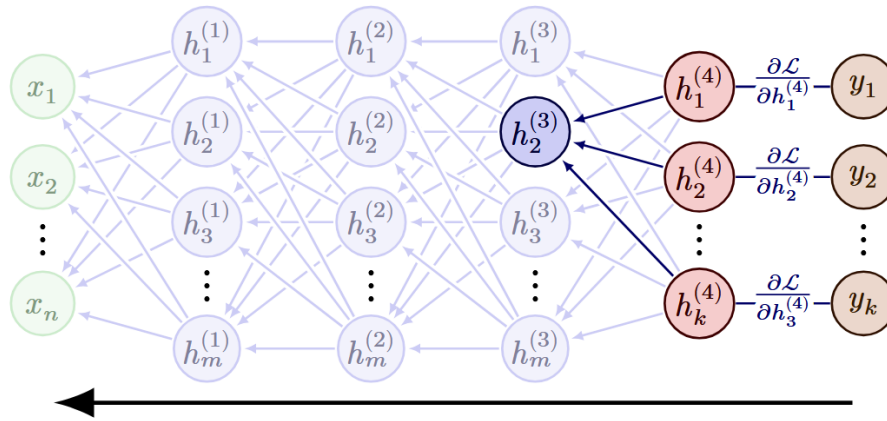
$$\frac{\delta g(f(x))}{\delta x} = g'(f(x)) \cdot f'(x)$$

Definition 6.4 (Backpropagation)

An algorithm that leverages the chain rule of calculus to efficiently compute the (stochastic) gradients of the loss with respect to every weight in the network. These gradients are then used to update the weights via optimization techniques such as SGD.

$$w_{i,j}^{(l)} \leftarrow w_{i,j}^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}}$$

- **Forward Pass:** The input is fed through the network to compute the loss, and all intermediate node activations are stored in memory.
- **Backward Pass:** Gradients are computed starting from the output layer and propagating backwards. Computations from later layers are reused for earlier layers to reduce computational complexity.
- **Memory Requirements:** Training requires roughly three times more memory than inference because it must store model parameters, intermediate activations, and the gradients simultaneously.



$$\frac{\partial \mathcal{L}}{\partial h_2^{(3)}} = \sum_{j=1}^k \frac{\partial \mathcal{L}}{\partial h_j^{(4)}} \frac{\partial h_j^{(4)}}{\partial h_2^{(3)}}$$

6.5 Regularization of Neural Networks

Adding regularization constrains the capacity of the model, which effectively increases its bias and reduces its variance to prevent overfitting. There are multiple techniques one can choose from:

- **Capacity:** Number of parameters
- **Structural:** Neural Architecture Search (NAS), Skip Connections, ...
- **Ensembling:** Dropout
- **Early Stopping:** Stop training when validation loss begins to continuously increase
- **Implicit:** Batch Normalization, Layer Normalization
- **L1 (Weight Decay):** Encourages sparse weights.

$$J = \mathcal{L}(w; x, y) + \alpha \|w\|_1 = \mathcal{L} + \alpha \sum_k |w_k| \quad \rightarrow \quad \nabla_w J = \nabla_w \mathcal{L}(w; x, y) + \alpha \cdot \text{sign}(w)$$

- **L2 (Weight Decay):** Smoothly penalizes large weight values.

$$J = \mathcal{L}(w; x, y) + \frac{\alpha}{2} w^T w = \mathcal{L} + \frac{\alpha}{2} \|w\|_2^2 \quad \rightarrow \quad \nabla_w J = \nabla_w \mathcal{L}(w; x, y) + \alpha w$$

6.6 Convolutional Neural Networks (CNNs)

- **Sparse Connectivity:** Instead of fully connected dense layers, CNNs use small kernels (filters) that only interact with a local receptive field. This captures meaningful local features (like edges) and greatly reduces both memory footprint and the number of computations.
- **Parameter Sharing:** The exact same kernel weights are applied (slid) across the entire input image. This drastically reduces the parameter count and provides **translation-equivariance** (if a feature shifts in the input, it shifts equally in the output).
- By stacking multiple convolutional layers, the network can capture increasingly complex features as deeper layers look at larger aggregated portions of the original image.
- For a 2D input V and a Kernel K , the convolution operation is given by:

$$Z(i, j) = \sum_m \sum_n V(i + m, j + n) K(m, n)$$

Where $Z(i, j)$ is the value of the resulting feature map at position (i, j) .

7 Ensembles

Ensembles are aggregations of multiple base models (weak learners) used to make a final prediction / predictive model (e.g., by averaging their outputs). The goal is to improve accuracy, **reduce variance**, and enhance robustness compared to any individual model.

$$\hat{y} = \frac{1}{K} \sum_{k=1}^K \hat{f}^{(k)}(x)$$

7.1 Bagging (Bootstrap Aggregation)

Definition 7.1 (Bootstrapping)

A statistical method of generating multiple diverse datasets by randomly sampling **with replacement** from the original training data.

Definition 7.2 (Bagging)

A technique which involves training one separate prediction model on each of the bootstrapped datasets and aggregating their predictions. Bagging is primarily used to reduce the **variance** of a model without significantly affecting its bias.

Requirements: For the ensemble's overall error to be reduced, the individual base models must be **uncorrelated** and have **low bias**. If the model errors are highly correlated, the ensemble average does not help reduce the squared error.

7.2 Random Forests

While standard decision trees have low bias, they tend to be correlated when trained on similar data. **Random Forests** (an extension of Bagged Trees) decorrelate the trees by selecting a random subset of m features at every node, and only picking the best decision split from those m features. This increased diversity among trees leads to a stronger ensemble, improving generalization and reducing overfitting.

- **Uncertainty:** The variance of the ensemble's predictions can be used as a measure of uncertainty.
- **Out-of-Bag (OOB) Error:** Bagged ensembles produce an internal estimate of the test error using the data points that were not selected in a particular bootstrap sample.

7.3 Boosting

Boosting is an ensemble technique that sequentially trains weak, **high-bias** models, where each new model tries to fix the mistakes of all previous ones. The final ensemble is a weighted sum of the sub-models, resulting in a **low-bias** ensemble. Boosting sequentially adds one model at a time while keeping the previously trained models fixed.

$$F^{(K+1)}(x) = F^{(K)}(x) + \alpha f^{(K+1)}(x)$$

7.4 Gradient Boosting

Gradient Boosting is a generic framework that uses gradient descent in functional space to sequentially improve the ensemble. The loss function is linearly approximated using a first-order Taylor expansion. The next submodel $f^{(K+1)}$ is trained to regress on the negative gradients (also called pseudo-residuals) of the loss function with respect to the current ensemble's predictions.

$$z_n = -\frac{\partial \mathcal{L}(y_n, F^{(k)}(x_n))}{\partial F^{(k)}(x_n)}$$

$$f^{(k)} := \arg \min_f \sum_{n=1}^N (f(x_n) - z_n)^2$$

7.5 AdaBoost

AdaBoost is a special case of Gradient Boosting that uses an exponential instance-penalty loss. It maintains a weight w_i for each data point, indicating how hard it is to predict. Initially, all weights are $1/N$. After a submodel is trained, the weights of **misclassified** instances are heavily increased, forcing the next submodel to focus on them.

$$w_i^{(k+1)} := w_i^{(k)} \exp\left(\alpha_k \mathbb{I}\left(y_i \neq f^{(k)}(x_i)\right)\right)$$

The submodel's weight α_k in the final ensemble is determined by its weighted training error rate ϵ_k . Better models (lower error) receive exponentially higher voting weights (e.g. for $\epsilon_k = 0.5 \rightarrow \alpha_k = 0$):

$$\alpha_k := \ln \frac{1 - \epsilon_k}{\epsilon_k}$$

The final prediction then is a weighted majority vote (classification as +1 or -1):

$$F^{(K)}(x) = \text{sign} \left(\sum_{k=1}^K \alpha_k f^{(k)}(x) \right)$$

8 Gradient Boosted Decision Trees

GBDTs are an ensemble learning method that combines decision trees with boosting to improve predictive accuracy. Trees are added sequentially, with each new tree trained to minimize the residual errors of the previous ensemble.

8.1 Tree Prediction Model & Regularization

A decision tree partitions the feature space into T leaves and outputs a specific weight w_j for each leaf. Let $q(x_n)$ denote the function that maps an input instance to its corresponding leaf index. The prediction of the tree is simply the weight of that assigned leaf.

$$f(x_n) = w_{q(x_n)}$$

A tree overfits if it has too many steps (excessive number of leaves T) or excessively high jumps between steps (large leaf weights w). To prevent this, a regularization term $\Omega(f)$ is added to penalize both the total number of leaves (controlled by hyperparameter γ) and the magnitude of the leaf weights (controlled by L2 regularization hyperparameter λ).

$$\Omega(f) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^T w_j^2$$

8.2 Boosted Ensemble Objective & Taylor Approximation

Gradient Boosted Decision Trees (GBDTs) use an additive strategy, creating an ensemble where the prediction at step k is the previous ensemble's prediction plus the new tree's prediction.

$$\hat{y}_n^{(k)} = \hat{y}_n^{(k-1)} + f^{(k)}(x_n)$$

To find the optimal k -th tree, the algorithm minimizes the loss function over all instances plus the regularization term of the new tree.

Because optimizing the loss function directly at each iteration of boosting is expensive and can lead to unstable updates, GBDTs approximate the loss using a **second-degree Taylor expansion**. This approximation relies on the first-order gradient G_n and the second-order Hessian H_n of the loss function with respect to the previous prediction.

$$\mathcal{L}_n^{(k)} \approx \mathcal{L}_n^{(k-1)} + \nabla_n f^{(k)}(x_n) + \frac{1}{2} H_n(f^{(k)}(x_n))^2$$

8.3 Optimal Leaves & Split Gain

By substituting the tree prediction w_j into the approximated objective, the optimal weight for any specific leaf j can be calculated using a closed-form solution.

The optimal weight for leaf j depends entirely on the sum of the gradients and Hessians of the instances assigned to that leaf, scaled by the regularization term λ .

$$w_j^* = -\frac{\sum_{n \in I_j} G_n}{\lambda + \sum_{n \in I_j} H_n}$$

To determine how to grow the tree (i.e., finding the best split), the algorithm calculates the **Gain** of splitting a leaf into a Left and Right child. The best split rule maximizes this gain.

$$Gain_j = \frac{1}{2} \left[\frac{(\sum_{Left} G_n)^2}{\lambda + \sum_{Left} H_n} + \frac{(\sum_{Right} G_n)^2}{\lambda + \sum_{Right} H_n} - \frac{(\sum_{Parent} G_n)^2}{\lambda + \sum_{Parent} H_n} \right] - \gamma$$

Stopping Condition: The tree stops growing a specific branch if the maximum split gain is less than or equal to zero (which is inherently controlled by the complexity penalty γ).

8.4 Properties & Algorithmic Complexity

- **Complexity:** For an ensemble of k trees with maximum depth $\log(T)$ and M features across N data points, the overall training complexity is $O(kMN \log(T))$.
- **Pros:** Extremely fast, works right out-of-the-box for tabular data without heavy preprocessing, handles categorical features natively, allows arbitrary loss functions, and produces low-bias and low-variance models.
- **Cons:** Less flexible than Neural Networks (e.g., cannot easily do multi-task learning or handle complex unstructured data), possesses less interpretability than a single decision tree, and has limited scalability compared to mini-batch NNs.

9 Hyperparameter Optimization

9.1 Principles & Validation

While model parameters are learned to minimize the loss on a training dataset, **hyperparameters** (which control model complexity, learning characteristics, and optimization choices) must be tuned externally.

The ultimate goal of Hyperparameter Optimization (HPO) is to find the configuration λ^* that minimizes the response function $f(\lambda, D)$ (e.g., the validation loss).

- **Single-Split Validation:** The data is divided into disjoint training, validation, and test sets. The model is tuned on the validation set, and the final model is retrained on the combined training and validation sets before testing.
- **K-Fold Cross Validation:** The data is partitioned into k disjoint folds. The evaluation is averaged over k runs, using a different fold for validation each time, to yield a more robust performance estimate.

9.2 Black-Box Search

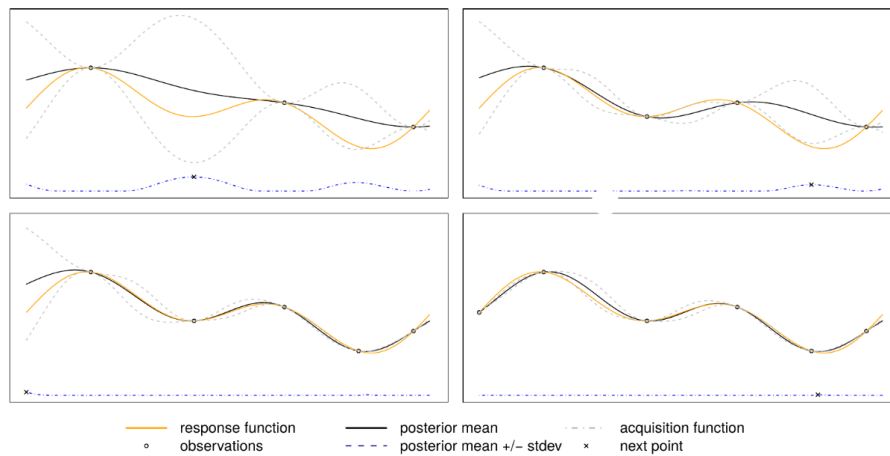
- **Grid Search:** Exhaustively evaluates configurations on a manually specified, fixed grid. It is generally inefficient because it wastes computational resources evaluating unimportant hyperparameter dimensions.
- **Random Search:** Randomly samples hyperparameter configurations. It typically outperforms Grid Search because it explores a larger number of unique values for the most important parameters.

9.3 Sequential Model Based Optimization (SMBO)

Definition 9.1 (Sequential Model Based Optimization)

Optimization of f is costly as it involves many runs of the learning algorithm. SMBO, also known as Bayesian Optimization, is an iterative technique that builds a surrogate model $\Psi \approx f$ based on previous observation history to cheaply approximate the true, computationally expensive response function.

1. Learn a surrogate (approximated) model on known observations $\Psi \approx f$
2. Query Ψ for regions Λ where **expected improvement is high**
3. Evaluate f in these regions by running the learning algorithm



- **Gaussian Processes (GP)**: Frequently used as the surrogate model because they provide both a predicted mean $\mu(\Psi(\lambda))$ and a variance/uncertainty estimate $\sigma^2(\Psi(\lambda))$ for unseen configurations.
- **Acquisition Function $a(\lambda, \Psi(\lambda))$** : A mathematical function used to query the next hyperparameter candidate. It balances **exploration** (searching regions of high uncertainty) and **exploitation** (searching regions with a good predicted mean).
- **Expected Improvement (EI)**: An acquisition function (AF) that computes the standardized expected improvement over the currently known minimum response.
- **Lower Confidence Bound (LCB)**: AF typically used for regression or minimization tasks (like error/loss).

$$LCB(\lambda) = -\mu(\Psi(\lambda)) + \beta\sigma(\Psi(\lambda))$$

- **Upper Confidence Bound (UCB)**: AF used for classification or maximization tasks (e.g. accuracy).

$$UCB(\lambda) = \mu(\Psi(\lambda)) + \beta\sigma(\Psi(\lambda))$$

Definition 9.2 (Mean and Variance)

Mean $\mu(\Psi(\lambda))$ represents the predicted performance of a hyperparameter λ . The calculation uses the kernel matrix K , which captures similarity between previous hyperparameter evaluations to make an estimate based on past observations. Used for **exploitation** in the AF.

Variance $\sigma^2(\Psi(\lambda))$ quantifies the uncertainty in the prediction. Used for **exploration** in the AF.

Note: The Kernel assumes that the more similar two hyperparameters are (i.e. the closer they are in the search space), the more correlated their predicted performances will be ($k(\lambda, \lambda') \approx 1$).

9.4 Gray-Box Search

Gray-box optimization significantly speeds up HPO by exploiting partial knowledge of the learning process (e.g., stopping poorly performing learning curves early, or rapidly evaluating models on smaller data subsets).

Definition 9.3 (Successive Halving)

Evaluates many configurations on a very small budget, then successively halves the number of surviving configurations while proportionally increasing their allocated training budget.

The Budget Problem: A major issue with Successive Halving is that evaluations at small budgets are often a noisy proxy for full-budget performance (low rank correlation).

Definition 9.4 (Hyperband)

An algorithm that mitigates the risks of Successive Halving by randomly trying different initial budgets and different numbers of configurations. It perfectly balances exploring many configurations (worse proxy correlation) against exploring fewer configurations at higher budgets (better proxy correlation).

Characteristics:

- Hyperband yields massive speedups and performs exceptionally well early in the search compared to Random Search.
- Bayesian Optimization (SMBO) is slower to start but performs very well in the later stages of optimization.
- **BOHB (Bayesian Optimization & Hyperband):** A state-of-the-art hybrid algorithm that combines the rapid early exploration of Hyperband with the strong final convergence of Bayesian Optimization.

10 Fighting Overfitting

10.1 Dropout

Definition 10.1 (Dropout)

A regularization technique for NNs that acts like bagging by training an ensemble of random subnetworks. It achieves this by randomly dropping out (multiplying by zero) input and hidden nodes during each training step.

How it works:

1. For each mini-batch, binary drop-out masks μ are sampled from a Bernoulli distribution (e.g., probability $p = 0.8$ to keep an input unit, and $p = 0.5$ to keep a hidden unit).
2. **Forward Pass:** The output of a layer is element-wise multiplied by its corresponding mask.

$$h^{(1)} = g^{(1)}(w^{(1)T}(x \odot \mu^{(0)}) + b^{(1)})$$

3. **Backward Pass:** Gradients are multiplied by the exact same masks that were sampled and stored during the forward pass.
4. **Inference:** To ensure the expected value of the activations remains the same during testing (when no nodes are dropped), the weights are scaled by the keep probability p . In practice, DL libraries apply inverse scaling ($1/p$) during training so no adjustments are needed at inference.

10.2 Structural Regularization

- **Residual Networks:** Introduce skip connections across layers (e.g., $\mathcal{F}(x) + x$). This allows the network to easily learn identity functions and helps gradients flow backward without vanishing, enabling the training of much deeper networks.
- **Shake-Shake:** Applied to residual blocks with multiple branches. It randomly weights the parallel paths for each mini-batch using a random variable $\alpha \in \mathcal{U}(0, 1)$. Different weights are sampled for the forward and backward passes, adding noise to the gradients which improves generalization.

10.3 Data Augmentation

Data augmentation expands the training set by applying label-preserving transformations to the inputs, creating modified versions of the same sample at different epochs.

- **Cutout:** Randomly sets a $k \times k$ square patch of an image to zero (black).
- **Mixup:** Creates new "Frankenstein" instances by taking a linear interpolation of two existing inputs and their corresponding labels, using a random factor $\lambda \in [0, 1]$.

$$\tilde{x} := \lambda x^{(a)} + (1 - \lambda)x^{(b)}$$

- **Cutmix:** Extends Mixup by swapping patches between two images using a random binary spatial mask.
- **Trainable Augmentation (AutoAugment):** Instead of manually engineering policies, a Recurrent Neural Network (RNN) controller learns the optimal augmentation strategy (operation types, probabilities, and magnitudes) via Reinforcement Learning, using the validation accuracy of a child network as the reward signal.

10.4 Regularization Cocktails

A single regularizer is rarely optimal. Instead, modern approaches combine a set of K different regularizers (e.g., Dropout, Mixup, Weight Decay, Batch Normalization) into a "**Cocktail**". The hyperparameters of all individual regularizers within the cocktail are jointly optimized using HPO techniques.

Empirically, fine-tuned cocktails of regularizers perform substantially better than any singular regularizer applied in isolation.

11 Error Measures

11.1 Binary Classification

A binary classification model predicts the probability $p(y_n = 1 | x_n) \approx f(x_n; \theta)$. The final estimated target $\hat{y}_n$ is determined by applying a threshold ϵ to the predicted probability. The results yield a confusion matrix.

Definition 11.1 (Misclassification Rate (MCR))

The ratio of incorrect predictions to the total number of predictions, which can be expressed mathematically as a sum of 0/1 error terms (0/1 step function is not differentiable, so does not work with SGD).

$$MCR(y, \hat{y}) = \frac{FP + FN}{TP + TN + FP + FN} = \frac{1}{N} \sum_{n=1}^N \mathbb{I}[y_n \neq \hat{y}_n]$$

Important: MCR is a highly misleading metric for imbalanced datasets (e.g., rare disease detection) because predicting only the majority class yields a high accuracy while completely failing the actual task.

Definition 11.2 (Proxy Loss)

A differentiable function that approximates the true, non-differentiable error metric. Minimizing the proxy loss should roughly lead to the same optimal parameters as minimizing the true metric.

Example: BCE acts as a smooth proxy for 0/1 MCR error.

11.2 Advanced Classification Metrics

- **Recall (True Positive Rate)**: Ratio of correctly detected positive cases out of all actual positive cases.

$$TPR = \frac{TP}{TP + FN}$$

- **False Positive Rate (FPR)**: Ratio of false alarms out of all actual negative cases.

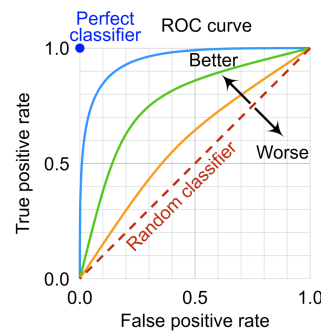
$$FPR = \frac{FP}{TN + FP}$$

- **Receiver Operating Characteristic (ROC)**: A curve that plots TPR against FPR across all possible decision thresholds $\epsilon \in [0, 1]$.
- **AUC**: The Area Under the ROC Curve, providing an aggregate measure of performance across all thresholds. It is maximized by correctly ranking positive instances higher than negative ones.
- **Precision**: The ratio of actual positive cases out of all cases that the model predicted as positive.

$$Precision = \frac{TP}{TP + FP}$$

- **F1 Score**: The harmonic mean of Precision and Recall. It is heavily used for imbalanced datasets. Because F1 is non-differentiable and non-decomposable, a common workaround is to oversample the minority class, train the model using a proxy loss, and then tune hyperparameters to explicitly maximize the validation F1 score.

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$



11.3 Regression Error Measures

Regression error measures are naturally differentiable and do not require proxies.

- **Scale-dependent errors**: Include Mean Absolute Error (MAE) and Root Mean Square Error (RMSE).

$$RMSE = \sqrt{\frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2}$$

- **Scale-independent (percentage) errors**: Include Mean Absolute Percentage Error (MAPE). However, MAPE becomes undefined or extreme when the true target $y_n \approx 0$.

$$MAPE = \frac{1}{N} \sum_{n=1}^N \left| \frac{y_n - \hat{y}_n}{y_n} \right|$$

- **Mean Absolute Scaled Error (MASE)**: Fixes the instability of MAPE by scaling the error using the average error of a naive baseline model instead of the true target value.

11.4 Ranking & Information Retrieval

The goal in ranking is to learn a function that outputs relevance scores $s_i = f(x_i)$ for documents matching a specific query q , such that the predicted order matches the true relevance labels l_i .

- **Discounted Cumulative Gain (DCG@K)**: A measure that sums the true relevances of the top K ranked documents, heavily penalizing relevant documents that are placed lower in the ranking.

$$DCG@K = \sum_{i=1}^K \frac{2^{l_i} - 1}{\log_2(i + 1)}$$

- **Normalized DCG (NDCG@K)**: DCG scales unboundedly depending on the query. NDCG normalizes it to a $[0, 1]$ range by dividing the predicted DCG by the Ideal DCG (IDCG), which is the DCG obtained by a perfectly sorted ranking.

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

Definition 11.3 (Pairwise Rank Loss)

A technique that reframes the ranking problem by estimating the probability that any given pair of documents (i, j) is correctly ranked relative to each other.

To train a pairwise ranking model, we define a target state $S_{i,j}$ (1 if document i is more relevant, -1 if less, and 0 if equal) and optimize the parameters to minimize the pairwise log loss.

$$\mathcal{L} * i, j = \frac{1}{2} (1 - S * i, j) (s_i - s_j) + \log(1 + e^{-(s_i - s_j)})$$

Method	Tasks		Complexity ¹		Pros	Cons
	Class. ¹	Regr.	Train / Fit	Inference		
SVM	B	—	$O(f n^2)$	$O(f)$	⊕ Works well when there is a clear margin ⊕ Effective in high dimensions ⊕ Good for $f > n$ ⊕ Memory efficient	⊖ Can't handle overlapping classes ⊖ Bad for large datasets ⊖ Not great for noisy data ⊖ Choosing a good kernel is hard
Decision Trees	B, MC	✓	$O(f n \log n)$	$O(\log n)$	⊕ Interpretable ⊕ Scalable for large datasets ⊕ Flexible framework ⊕ Handle categorical features well	⊖ Prone to overfitting ⊖ Deterministic
Neural Networks	B, MC	✓	variable	variable	⊕ Universal, flexible framework ⊕ Suitable for complex tasks ⊕ Multitask	⊖ Require large amounts of data to not overfit ⊖ Computationally costly to train ⊖ Difficult to interpret
GBDT	B, MC	✓	$O(e f n \log(n))$	$O(e \log(n))$	⊕ Same as Decision Trees ⊕ Works with arbitrary Loss functions ⊕ ↓↓ bias / ↓ variance	⊖ Less flexible than Neural Networks ⊖ Less interpretability ⊖ Limited scalability

12 Algorithms

SGD

Input: Data $\mathcal{D} = \bigcup_{r=1}^R \mathcal{D}_r$, Learning rate $\eta \in \mathbb{R}_+$, Iterations $I \in \mathbb{N}_+$.
Ensure: $w \in \mathbb{R}^K$

```

1  $w \sim \mathcal{N}(0, \sigma^2)$ 
2 for  $1, \dots, I$  do
3   foreach  $r \in \{1, \dots, R\}$  in random order do
4     Forward pass using  $x = \pi_x(\mathcal{D}_r)$  to compute  $J_r$ 
5     Backpropagation to compute  $\frac{\partial J_r}{\partial w}$ 
6      $w \leftarrow w - \eta \frac{\partial J_r}{\partial w}$ 
7 return  $w$ 

```

Learning a Perceptron Model (SVM)

Input: Data $x \in \mathbb{R}^{N \times M}$, $y_i \in \{-1, 1\}^N$, Learning rate $\eta \in \mathbb{R}_+$
Ensure: $w \in \mathbb{R}^M, w_0 \in \mathbb{R}$

```

1  $w \sim \mathcal{N}(0, \sigma^2)_M, w_0 \sim \mathcal{N}(0, \sigma^2)$  // Random initial hyperplane
2 errors  $\leftarrow 1$ 
3 while errors  $> 0$  do
4   errors  $\leftarrow 0$ 
5   for  $i = 1, \dots, N$  do
6     if  $y_i \neq \text{sign}(w^\top x_i + w_0)$  then
7       errors  $\leftarrow$  errors  $+ 1$ 
8        $w \leftarrow w + \eta y_i x_i$ 
9        $w_0 \leftarrow w_0 + \eta y_i$ 
10 return  $w, w_0$ 

```

CART

Data: Params: Set $\mathcal{D} := \{(x_i, y_i)\}_{i=1}^{|\mathcal{D}|}$, Node n , Depth ℓ
Hyper-params: Max Depth: L , Min Instances for Split: δ

```

1 if  $|\mathcal{D}| > \delta \wedge \ell \leq L$  then
2    $(x_{:,j}, v) := \text{SearchSplit}(\mathcal{D})$  // Find best split
3    $n \leftarrow (x_{:,j} \leq v, \text{yes/no?})$  // Decision node
4    $\mathcal{D}^{(L)} := \{(x, y) \in \mathcal{D} \mid x_{:,j} \leq v\}$ 
5    $\text{CART}(\mathcal{D}^{(L)}, n_{\text{left}}, \ell + 1)$  // Create left sub-tree
6    $\mathcal{D}^{(R)} := \{(x, y) \in \mathcal{D} \mid x_{:,j} > v\}$ 
7    $\text{CART}(\mathcal{D}^{(R)}, n_{\text{right}}, \ell + 1)$  // Create right sub-tree
8 else
9    $n \leftarrow \hat{y} = \text{aggregate}(\pi_y(\mathcal{D}))$  // Leaf node

```

Call initially: $\text{CART}(\mathcal{D}, n_{\text{root}}, 1)$

Random Forest

```

1 for  $k = 1, \dots, K$  do
2   Draw bootstrap  $\mathcal{D}^{(k)}$  with replacement from  $\mathcal{D}^{(\text{Train})}$ 
3   Train tree  $f^{(k)}(x)$  through CART with a randomized split search for every node:
     a) Select  $m$  features at random from the  $M$  features
     b) Pick the best decision split among the  $m$  features
4  $\hat{f}(x) = \frac{1}{K} \sum_k \hat{f}^{(k)}(x)$ 

```

K-Fold Cross-Validation

```

1 Require  $D$  with  $N$  data points,  $2 \leq k \leq N$ 
2 Split  $D$  into  $k$  disjoint  $D_1, \dots, D_k$ 
3 for  $i = 1$  to  $k$  do
4    $D^{\text{test}} := D_i$                                      // Test set
5    $D^{\text{val}} := D_j, j \sim \mathcal{U}(1, \dots, k) \wedge j \neq i$            // Validation set
6    $D^{\text{train}} := D \setminus [D^{\text{test}} \cup D^{\text{val}}]$                // Train set
7   Apply single-split protocol and set  $\mathcal{L}^{(i)} := \mathcal{L}(\hat{y}_{\lambda^*}^{\text{opt}}, D^{\text{test}})$ 
8 return  $\frac{1}{k} \sum_{i=1}^k \mathcal{L}^{(i)}$ 

```

SMBO

```

1 Require: Initial observation history  $\mathcal{H}$ , surrogate model  $\Psi$ , acquisition function  $a$ 
   Result: Best hyperparameter configuration  $\lambda^*$ 
2 Initial evaluations  $\mathcal{H} := \{(\lambda_1, f(\lambda_1, D)), \dots, (\lambda_K, f(\lambda_K, D))\}$ 
3 for  $t = 1$  to  $T$  do
4   Fit  $\Psi$  to  $\mathcal{H}$ 
5    $\lambda^{\text{new}} := \arg \max_{\lambda \in \Lambda} a(\lambda, \Psi(\lambda))$ 
6   Evaluate  $f(\lambda^{\text{new}}, D)$ 
7    $\mathcal{H} := \mathcal{H} \cup \{(\lambda^{\text{new}}, f(\lambda^{\text{new}}, D))\}$ 
8 return  $\lambda^* = \arg \min_{\lambda} f(\lambda, D)$ 

```

Note: f can also be classification accuracy and the problem becomes a maximization, i.e. $\lambda^* = \arg \max_{\lambda} f(\lambda, D)$